Blatt 06: Interpreter

Carsten Gips, BC George (HSBI)

1 Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung eines Tree-Walking-Interpreters für eine Lisp-artige Sprache.

2 Methodik

Sie haben in Blatt 04 eine Grammatik für die Lisp-artige Sprache definiert und darauf aufbauend einen Lexer und Parser implementiert, d.h. Sie können für eine Eingabe eines Programms mit S-Expressions den entsprechenden AST erzeugen.

Korrigieren Sie die Codebasis von Blatt 04, falls notwendig. Ergänzen Sie sodann einen Tree-Walking-Interpreter und eine REPL für die Lisp-artige Sprache.

Es ist empfehlenswert, den Interpreter dreistufig zu realisieren:

1. Einlesen aus einer Datei mit Lisp-Code und Parsen der Inhalte
2. Aufbauen der Symboltabelle und Durchführung der semantischen Analyse
3. Ablaufen des Parse-Tree/AST und Auswerten der Ausdrücke (Interpretation)

3 Sprachdefinition

Siehe Blatt 04.

4 Aufgaben

4.1 A6.1: Semantische Analyse (2P)

1. Erstellen Sie zunächst einige Programme in der Zielsprache. Diese sollten von einfachsten Ausdrücken bis hin zu komplexeren Programmen reichen. Definieren Sie beispielsweise eine Funktion, die rekursiv die Länge einer Liste berechnet.

   Definieren Sie neben gültigen Programmen auch solche, die in der semantischen Analyse zurückgewiesen werden sollten. Welche Fehlerkategorien könnte es hier geben?

2. Führen Sie die semantische Analyse durch: Sind alle Symbole bekannt, passen die Scopes?

4.2 A6.2: Tree-Walking-Interpreter (3P)

Bauen Sie einen Tree-Walking-Interpreter in Ihr Projekt ein.

Realisieren Sie die eingebauten Funktionen `print` und `str` dabei als *native* Funktionen. Realisieren Sie `list`, `nth`, `head` und `tail` sowie `def`, `let`, `defn`, **`do`** und die Operatoren und die Kontrollstrukturen geeignet.

Lösen Sie die als "*syntactic sugar*" bezeichneten Ausdrücke auf und transformieren Sie den AST entsprechend: `(+ 1 2 3 4)` soll zu `(+ (+ (+ 1 2)3)4)` umgeformt werden. Analog für die anderen Operatoren der Sprache (Vergleiche, Arithmetik).

Achten Sie auf die Datentypen. Die Typen von Variablen etc. sind erst zur Laufzeit bekannt und müssen dann passen.

Lesen Sie den zu interpretierenden Code aus einer Datei ein.

Testen Sie Ihren Interpreter mit Ihren Beispielprogrammen.

4.3 A6.3: Interaktiver Interpreter (3P)

Bauen Sie eine *REPL* ein, d.h. geben Sie nach dem Start des Interpreters einen Prompt aus und verarbeiten Sie die Eingaben interaktiv. Wie müssen Sie hier mit der Symboltabelle umgehen?

4.4 A6.4: Auswirkungen der Grammatik auf den Interpreter (2P)

Vergleichen Sie ihre eigene Grammatik mit den beiden Grammatiken (MiniLispA, MiniLispB).

Welche Auswirkungen hat die Grammatik auf den Interpreter? Machen Sie ein Gedankenexperiment: Überlegen Sie, was Sie alles in Ihrer Implementierung ändern müssten, wenn Sie die jeweils andere Grammatik-Variante nutzen würden.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 24964bc (homework: finalize B06 (#378), 2025-10-03)